

LoKA: Low-precision Kernel Applications for Recommendation Models At Scale

Liang Luo, Yinbin Ma, Quanyu Zhu, Vasiliy Kuznetsov, Yuxin Chen, Neng Shi,
Jian Jiao, Jiecao Yu, Buyun Zhang, Tongyi Tang, Xiaohan Wei, Yanli Zhao, Zeliang Chen,
Yuchen Hao, Venkatesh Ranganathan, Sandeep Parab, Yantao Yao, Maxim Naumov,
Chunzhi Yang, Shen Li, Ellie Wen, Wenlin Chen, Santanu Kolay, Chunqiang Tang

Meta AI

Menlo Park, CA, USA

{liangluo, yinbin, qyz, vasilii, yuxinc, jianj, jiecaoyu, buyunz, tongyitang, ubimeteor,
yanlizhao, zlc, haoyc, verangan, sandeeparab, yantao, mnaumov, zorrer,
shenli, ellie.wen, wenlinchen, skolay, tang}@meta.com

Abstract—Recent GPU generations deliver significantly higher FLOPs using lower-precision arithmetic, such as FP8. While successfully applied to large language models (LLMs), its adoption in large recommendation models (LRMs) has been limited. This is because LRMs are numerically sensitive, dominated by small matrix multiplications (GEMMs) followed by normalization, and trained in communication-intensive environments. Applying FP8 directly to LRMs often degrades model quality and prolongs training time. These challenges are inherent to LRM workloads and cannot be resolved merely by introducing better FP8 kernels. Instead, a system-model co-design approach is needed to successfully integrate FP8. We present LoKA (Low-precision Kernel Applications), a framework that makes FP8 practical for LRMs through three principles: profile under realistic distributions to know where low precision is safe, co-design model components with hardware to expand where it is safe, and orchestrate across kernel libraries to maximize the gains. Concretely, LoKA Probe is a statistically grounded, online benchmarking method that learns activation and weight statistics, and quantifies per-layer errors. This process pinpoints safe and unsafe, fast and slow sites for FP8 adoption. LoKA Mods is a set of reusable model adaptations that improve both numerical stability and execution efficiency with FP8. LoKA Dispatch is a runtime that leverages the statistical insights from LoKA Probe to select the fastest FP8 kernel that satisfies the accuracy requirements. Deployed on production LRMs that serve billions of users with advertising recommendations at a major social media company, LoKA delivers up to 20% higher training throughput and 40% faster inference on heterogeneous GPUs (H100, B200, GB200, MI300X and MI350X) in production environment, with no quality loss, turning FP8 from a model-quality risk into a reliable performance lever for LRMs at scale.

I. INTRODUCTION

Lower-precision numerics, such as FP8 and FP4, have been major drivers of GPU performance improvements due to their efficient use of chip area and power. As shown in Table I, the B200’s TF32 FLOPs is 7× that of the A100, while its FP8 and FP4 FLOPs are 29× and 58× the A100’s TF32 FLOPs, respectively. To fully leverage the high FLOPs of lower-precision numerics, ML models must adopt them effectively without sacrificing quality.

While recent work [19], [35] demonstrated that FP8 can be applied effectively to large language models (LLMs), to

TFLOPs	TF32	BF16	FP8/INT8
A100 [57]	156	312	624
H100 NVL [59]	418	840	1,671
B200 HGX [58]	1,100	2,250	4,500

TABLE I
NVIDIA GPU DENSE PEAK PERFORMANCE.

our knowledge, no prior work has achieved successful FP8 training for large recommendation models (LRMs) at scale. LRMs are critical industrial workloads that power applications such as advertising, short-video, and news-feed ranking. They predate LLMs and differ substantially in architecture and numerical behavior. Applying FP8 to LRMs is particularly challenging: on a state-of-the-art framework, TorchAO [60], we observe 1.3× slowdown and up to 2.5% relative log loss degradation (note that 0.02% relative log loss is considered significant in production) when applying FP8 training to a representative production recommendation model, due to the following characteristics.

First, LRMs operate under tight quality constraints—even minor degradations in model accuracy are unacceptable, leaving little room for approximation. Second, LRMs are architecturally heterogeneous. Unlike LLMs’ relatively uniform Transformer [67] stacks, LRMs combine wide ensembles [64], deep hierarchical stacking [78], [79], and specialized interaction modules [64], each exhibiting distinct numerical sensitivities. Third, LRMs exhibit low arithmetic intensity despite LLM-scale FLOPs complexity [76]. They consist primarily of many small GEMMs followed immediately by normalization layers, where quantization overhead can dominate and often negate the benefits of low-precision execution.

These challenges cannot be solved merely by introducing better low-precision kernels, as the fundamental problems are inherent to how LRMs are structured and trained. What is required is a system-model co-design approach that can diagnose where low precision is viable, modify model components to remain stable under reduced precision, and select the right kernels at runtime to maximize efficiency and maintain quality.

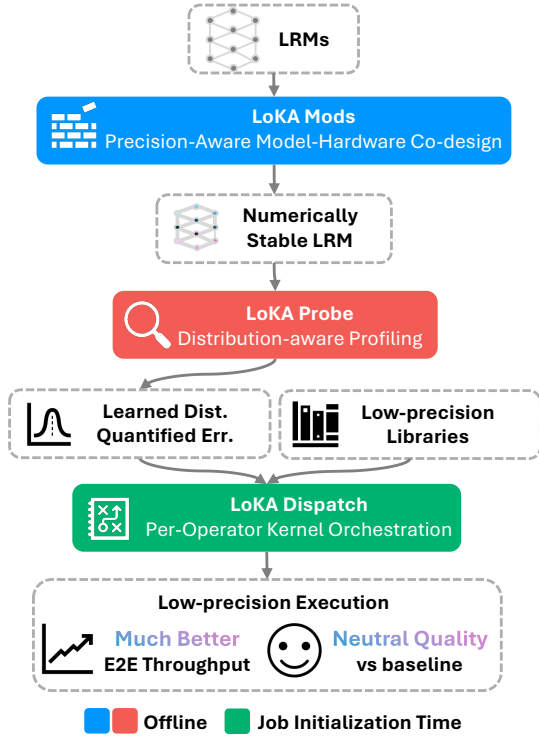


Fig. 1. LoKA Overview

We present LoKA (Low-precision Kernel Applications), a framework designed to unlock the benefits of FP8 and emerging precisions for large-scale recommendation models.

LoKA is built on top of three principles (Figure 1):

- 1) **Distribution-Aware Profiling — Know where low precision is safe.** Standard library benchmarks use synthetic inputs and overestimate low-precision readiness. Real workload distributions, especially confounded by complex LRMs, can be heavy-tailed, correlated, or non-stationary. These expose quantization errors that random tensors miss. Any low-precision adoption should begin by profiling under realistic distributions to identify per-operator risk. To that end, LoKA introduces *LoKA Probe*, a statistically grounded benchmarking method that identifies which layers in an LRM can safely operate at low precision efficiently.
- 2) **Precision-Aware Model-Hardware Co-design — Expand where low precision is safe.** When profiling reveals that certain operators are numerically unsafe or inefficient under low precision, the solution is not to fall back to high precision universally but to co-design model components with hardware execution constraints. This means strategically modifying operations so that they are simultaneously more numerically stable and more hardware-efficient. We propose *LoKA Mods*, a collection of redesigned modules that improve numerical stability under low-precision execution, reducing the risk of divergence or quality degradation.
- 3) **Per-Operator Kernel Orchestration — Maximize the gains from low precision.** No single low-precision library or recipe dominates across all operator shapes and hardware. Treating each operator as an independent optimization

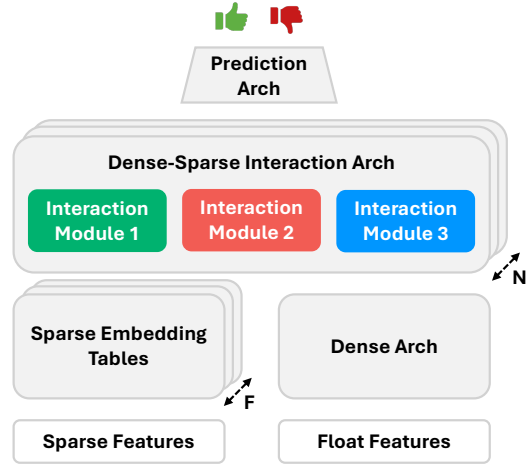


Fig. 2. A typical model architecture of a LRM.

problem — selecting the fastest implementation that satisfies an accuracy constraint — consistently outperforms any uniform policy. LoKA adopts *LoKA Dispatch*, a unified runtime mechanism that integrates multiple low-precision libraries and dynamically selects the fastest kernel that satisfies both accuracy and throughput constraints.

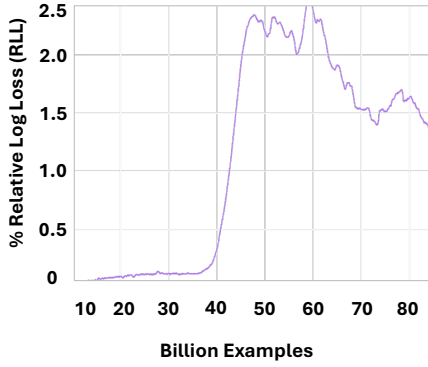
Together, these principles ensure LoKA remains effective across a wide range of hardware and beyond specific workloads. To demonstrate this, we leverage actual training data, comprising thousands of features, along with realistic batch sizes and production-scale model configurations and assess LoKA across three state-of-the-art recommendation model families integrating a wide range of industrial and open-source architectures [5], [53], [67], [70], [79], [80]: Wukong [78], Interformer [74] and External Large FM (ELFM) [43]. Across diverse scales and hardware generations including Nvidia H100, B200 and GB200 as well as AMD MI 300X and 350X clusters, LoKA consistently delivered significant end-to-end gains. It enabled up to 20% faster training throughput and 40% faster inference throughput in our hyperscale production environment, all while preserving model quality. LoKA was successfully launched to flagship models, serving billions of users at a major social media company for both training and inference. These results demonstrate that low-precision computation, when applied through a systematic framework, can serve as a practical and reliable accelerator for production-scale recommendation systems.

II. LOW PRECISION COMPUTATION FOR RECOMMENDATION MODELS

This section examines the unique challenges of applying low-precision training to recommendation models, highlighting key differences from language models that make direct adoption of existing techniques ineffective.

A. Large Recommendation Models

Modern recommendation models, illustrated in Figure 2, process diverse inputs to predict user-item interactions. They



BF16 Throughput	FP8 Throughput	FP8 Slowdown	Relative Log Loss (%)
520K/s	395K/s	1.32×	2.5

Fig. 3. Significant Relative Log Loss (top) and throughput (bottom) degradation are observed when training a Wukong model in FP8 low precision.

comprise four main components:

- **Sparse embedding tables** store dense vectors for discrete features like user IDs and item categories. These tables contain most model parameters and are distributed across GPUs using techniques like embedding sharding.
- **Dense architecture** processes continuous numerical features through standard neural network layers.
- **Dense-sparse interaction modules** combine embeddings and dense features through stacked layers containing specialized interaction components including deep cross networks [70], multi-head attention [15], factorization machines [78], or ensembles [79] thereof. They dominate computational complexity in the LRM.
- **Prediction layers** generate final scores from features.

Like LLMs, LRMs follow scaling laws [7], [31], [33], [40], [78] and have reached comparable FLOPS complexity. However, unlike LLMs that process relatively static data requiring single training runs, LRMs must continuously adapt to evolving user preferences and new items through online training [30], making both training and inference performance critical.

Modern LRMs use hybrid training [50]: sparse embedding tables are sharded across GPUs [75], and are communicated via the AlltoAll collectives [48], [50], while dense components use Fully Sharded Data Parallel (FSDP) [81].

B. Low-precision Training and Inference

Low-precision approaches span several categories [32]. Quantization-Aware Training (QAT) simulates quantization effects during training while maintaining full-precision weights, improving accuracy but not training speed. Post-Training Quantization (PTQ) applies quantization to trained models without additional training, offering simplicity but typically larger accuracy loss. Post-Quantization Training (PQT) combines PTQ with fine-tuning to recover accuracy. Direct Low Precision (DLP) employs native low-precision training and inference, offering the greatest throughput benefits. DLP is the focus of LoKA, though it presents the greatest technical

challenges because it requires native low-precision throughout entire training and inference.

Despite successful FP8 adoption in LLMs, direct application to LRMs fails dramatically. To demonstrate this, we applied TorchAO [60], a widely-used PyTorch low-precision library to all linear layers (input/output features ≥ 128) in the state-of-the-art Wukong model using 64 H100 GPUs with tensorwise FP8 scaling.

Figure 3 shows the results: FP8 training incurs 1.3× slowdown and up to 2.5% relative log loss degradation. This failure stems from fundamental differences between LRMs and LLMs highlighted in §I.

	Normal Dist.	LRM Input Dist.
Pytorch BF16	0.03	0.04
TorchAO RW	0.47	0.53
DeepGEMM BW	0.49	0.56
FBGEMM RW	0.48	0.52

TABLE II
GEOMETRIC MEAN OF MERE OF 4 IMPORTANT LINEAR LAYERS IN A LRM COMPARED TO TF32 BASELINE FOR LOW-PRECISION KERNELS WITH DIFFERENT INPUT DISTRIBUTIONS.

C. Characterizing Low-Precision Quantization Overheads and Error in LRMs

Standard low-precision libraries optimize for accuracy and performance using random or fixed-distribution [18], [27], [66] tensor benchmarks and common GEMM shapes [60]. However, recent study [22] has demonstrated that at scale, models develop highly systematic outlier activations that severely disrupt standard quantization techniques and real LRM workloads deviate substantially from these test conditions.

We analyzed a four-layer production LRM with linear layers, comparing three state-of-the-art low-precision libraries—FBGEMM [41], TorchAO, and DeepGEMM [17]. For each library, we evaluated two input distributions: (1) standard normal (as used in typical benchmarks), and (2) learned distributions obtained from actual LRM training traces.

Using the learned distributions allows us to synthesize representative inputs and quantify how errors manifest on out-of-distribution data, the conditions that are rarely exercised by standard benchmarks. We enabled fast accumulation, sampled 50 input–weight pairs per layer, and computed average statistics to ensure significance. Only forward performance was tested for simplicity.

Error Analysis. For each linear layer, we measure the FP8 execution error relative to a TF32 reference using Mean Element-wise Relative Error (MERE):

$$\text{MERE}(\text{out}^{(M,N)}, \text{ref}^{(M,N)}) = \sum_m^M \sum_n^N \left| \frac{\text{out}_{m,n} - \text{ref}_{m,n}}{\text{ref}_{m,n}} \right|$$

MERE captures the average element-wise deviation of low-precision outputs from full-precision results.

Findings. Table II summarizes results: the geometric mean of MERE across layers increases by up to 15% when tested with realistic LRM distributions versus standard normal inputs. This

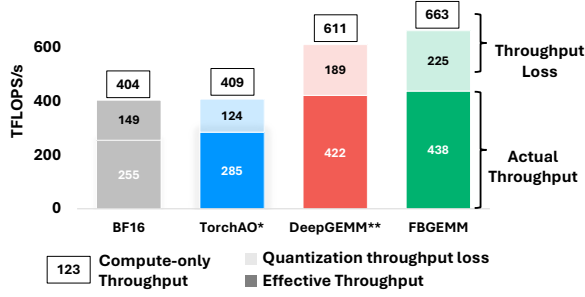


Fig. 4. Compute throughput ablation of low-precision kernels on representative LRM shapes. **/: Padding overheads not included in both TorchAO and DeepGEMM; torch.compile not available for DeepGEMM.

demonstrates that out-of-distribution activations significantly amplify quantization errors, precisely the failure mode not revealed by conventional random benchmarks.

Throughput Analysis We benchmarked 27 GEMM shapes from production LRMs, ranging from (2048, 256) @ (256, 768)¹ to (2048, 123200) @ (123200, 1024) on H100 GPUs. Figure 4 shows results separated into pure compute throughput versus end-to-end performance including quantization overhead. Key findings:

- End-to-end FP8 speedup over BF16 is limited to 1.6× on average.
- Maximum effective TFLOPS/s remains under 20% of hardware capacity.
- Quantization overhead consumes over 30% of end-to-end GEMM latency.
- Including memory allocation overhead (e.g., layout manipulation), FP8 can perform worse than BF16.

These results demonstrate that applying low-precision computation to LRMs requires addressing both accuracy degradation from realistic data distributions and performance overhead from quantization operations which are challenges that cannot be solved by library improvements alone.

D. The Deployment Status Quo of Low Precision Kernels

	Training	Inference
TF32	95%	0%
BF16/TF32	5%	-
FP16	-	99%
FP8	0	1% (PTQ)

TABLE III

PROPORTION OF MODELS TRAINED AND SERVED USING A SPECIFIC DATATYPE. - MEANS THIS CHOICE IS NOT APPLICABLE.

Real-world adoption of low-precision training and inference for LRMs remains limited. In a survey of top 500 Ads ranking LRMs at a large social-media company (Table III), we observed: in training, 95% of models run in high precision (TF32), 5% use mixed precision (BF16/TF32), and 0% train in FP8; in inference, 99% of models serve in FP16, with only 1% using FP8 via PTQ. These figures underscore the practical hurdles: numerical stability, small-GEMM quantization overheads, and

¹Denote a matmul of two tensors with shapes (2048, 256) and (256, 768)

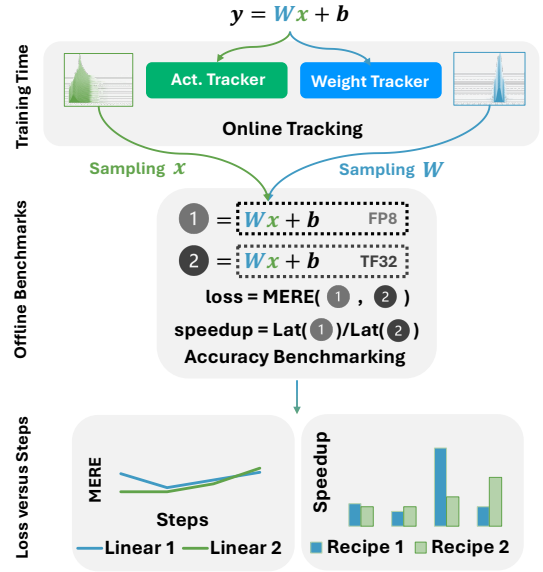


Fig. 5. LoKA Probe learns and stores necessary parameters online for offline accuracy benchmarks to derive statistically significant loss for each linear layer under low precision execution.

communication-dominated runtimes, and all these impede FP8 deployment at scale.

III. LoKA

As established in §I, the fundamental barriers to low-precision LRM training cannot be addressed by better kernels alone. They require a systematic approach guided by three principles: profiling under realistic distributions to know where low precision is safe, co-designing model components with hardware to expand where it is safe, and orchestrating across kernel libraries to maximize the resulting gains. LoKA instantiates each principle as a concrete component. LoKA Probe implements distribution-aware profiling (Principle 1). LoKA Mods realizes precision-aware model-hardware co-design (Principle 2). LoKA Dispatch provides per-operator kernel orchestration (Principle 3). We describe each below.

A. LoKA Probe: Distribution-Aware Profiling

The critical insight behind LoKA is that *standard low-precision testing fundamentally underestimates real-world quantization errors*. Existing libraries benchmark accuracy using random tensors (typically normal distributions) which fail to capture the complex statistical properties of actual LRM activations and weights. This leads to overly optimistic error estimates and explains why “battle-tested” low-precision kernels still cause training divergence in practice.

LoKA Probe addresses this by learning the true distributions of inputs and weights during training, then using these learned distributions for statistically significant offline accuracy (and throughput) assessment. This approach reveals quantization errors that random testing misses, enabling informed decisions about which layers can safely use low precision.

1) *Online Distribution Learning*: LoKA Probe operates in two phases: online learning during training and offline benchmarking for error analysis.

During training, LoKA Probe efficiently tracks the statistical properties of each linear layer’s inputs and weights without storing the actual tensors (which would be prohibitively expensive and prone to overfitting). Instead, it maintains compact statistical summaries that can recreate realistic distributions. LoKA Probe models distributions as multivariate Gaussians. For a 2D tensor T of shape (M, N) , LoKA Probe samples from

$$T \sim \mathbb{G}(\mu, \Sigma)$$

where $\mu \in \mathbb{R}^{M \times N}$ is the mean and $\Sigma \in \mathbb{R}^{MN \times MN}$ is the covariance matrix.

This Gaussian assumption is motivated by both theoretical and empirical observations: activations in large models, after layer normalization or RMS normalization, tend to concentrate around zero and exhibit light-tailed, symmetric statistics, which are well captured by a normal distribution. Moreover, under the central-limit effect, the pre-activation of each neuron aggregates many independent (or weakly correlated) input contributions, making its distribution approximately Gaussian even in nonlinear regimes.

Finally, LoKA’s objective is not to perfectly reproduce the long-tail statistics of activations but to approximate their *second-order structure* sufficiently for quantization-error analysis. Within this scope, Gaussian families provide analytically closed moments, stable parameter updates, and simple sampling rules, making them the pragmatic choice for large-scale, online distribution tracking.

However, while conceptually simple, storing the full covariance matrix ($M^2 N^2$ elements) is still intractable for large tensors. Therefore, we seek to significantly reduce LoKA Probe’s storage requirements.

Optimized Input Distribution Modeling Activations play a critical role in selecting layers to quantize [45]. For them, we exploit the independence of the batch dimension, a key property in recommendation models where cross-batch operators like BatchNorm are avoided to prevent information leakage². By treating the batch dimension as independent, it enables us to model only along the feature dimension, reducing storage from $O(M^2 N^2)$ to manageable $O(N^2)$.

Tracking the mean is trivially done in a streaming fashion. For variance, we implement a batched Welford tracker [13] that efficiently updates covariance online using only $O(N^2)$ memory, by decomposing the final covariance as follows.

Let the current batch be $X \in \mathbb{R}^{B \times K}$ (rows are samples, columns are features, therefore $B = M$ and $K = N$ using previous annotations). Define the batch mean $\mu_b \in \mathbb{R}^K$ and batch *scatter*:

$$S_b = (X - \mathbf{1}_B \mu_b^T)^T (X - \mathbf{1}_B \mu_b^T) \in \mathbb{R}^{K \times K}.$$

²Consider the case when the current (user, item) pair and the user’s future iteration (user, item in the future) both present in the same local batch, a batch norm would effectively use the user’s *future* behavior, which may already encode their current action in its input feature, to predict their current behavior, leading to catastrophic overfitting.

Given historical summaries $(n_{\text{old}}, \mu_{\text{old}}, \Sigma_{\text{old}})$, where Σ_{old} is the (unnormalized) scatter matrix, the merged quantities are

$$\begin{aligned} n_{\text{new}} &= n_{\text{old}} + B, \\ \delta &= \mu_b - \mu_{\text{old}}, \\ \mu_{\text{new}} &= \mu_{\text{old}} + \frac{B}{n_{\text{new}}} \delta, \\ \Sigma_{\text{new}} &= \Sigma_{\text{old}} + S_b + \frac{n_{\text{old}} B}{n_{\text{new}}} \delta \delta^T. \end{aligned}$$

The the unbiased (sample) covariance is

$$\Sigma = \frac{\Sigma_{\text{new}}}{n_{\text{new}} - 1} \quad (\text{for } n_{\text{new}} > 1).$$

Implementation notes. LoKA Probe accumulates Σ in higher precision (e.g., FP32).

Optimized Weight Distribution Modeling We cannot assume dimension independence for W (weight matrix), so the trick for input modeling does not apply. Instead, we model a weight matrix $W \in \mathbb{R}^{M \times N}$ with a matrix-normal distribution:

$$W \sim \mathcal{MN}(M, U, V), \quad \text{and} \quad \text{vec}(W) \sim \mathcal{N}(\text{vec}(M), V \otimes U),$$

where $U \in \mathbb{R}^{M \times M}$ is the row covariance and $V \in \mathbb{R}^{N \times N}$ is the column covariance. This reduces storage from $O(M^2 N^2)$ to $O(M^2 + N^2)$ while capturing essential correlations.

Let $W_c = W - M$ be the mean-centered weight. We maintain (U, V) online using a Kronecker-factor (flip-flop-style) update with exponential moving average (EMA). To avoid explicit matrix inverses, we use linear solves with (regularized) Cholesky factors.

Per-update (single minibatch) estimates:

$$\begin{aligned} \text{Solve } L_V L_V^T &= V + \varepsilon I, & \widetilde{W} &= W_c L_V^{-T}, \\ U' &= \frac{1}{N} \widetilde{W} \widetilde{W}^T, & (M \times M) \\ \text{Solve } L_U L_U^T &= U + \varepsilon I, & \widehat{W} &= L_U^{-1} W_c, \\ V' &= \frac{1}{M} \widehat{W}^T \widehat{W}, & (N \times N) \end{aligned}$$

EMA smoothing:

$$\begin{aligned} U'' &= m U + (1 - m) U', \\ V'' &= m V + (1 - m) V', \\ U &\leftarrow \frac{1}{2}(U'' + U''^T) + \varepsilon I, \\ V &\leftarrow \frac{1}{2}(V'' + V''^T) + \varepsilon I. \end{aligned}$$

Scale identifiability. Because $V \otimes U$ is invariant under $(U, V) \mapsto (cU, V/c)$ (where $\otimes$ is the Kronecker product [34], we renormalize to prevent drift using s for better numerical stability:

$$s = \frac{\text{trace}(U)}{M}, \quad U \leftarrow \frac{U}{s}, \quad V \leftarrow s V.$$

These updates allow us to avoid forming V^{-1} or U^{-1} explicitly. In practice we use small ε (e.g., $10^{-6} \times \frac{\text{trace}(U)}{M}$) and a momentum $m \in [0.9, 0.99]$ for stable online tracking.

To minimize overhead, LoKA Probe activates every 100 training iterations and asynchronously saves statistical parameters every 10,000 iterations, translating to a negligible ($\leq 1\%$) throughput overhead. This provides comprehensive coverage of distribution evolution throughout training while maintaining minimal performance impact.

Sampling learned distributions. Given the tracked statistics (μ, Σ) for activations and (M, U, V) for weights, LoKA Probe can synthesize representative inputs and weights for offline benchmarking.

(a) *Input sampling.* We draw a synthetic activation batch $T' \in \mathbb{R}^{B \times K}$ by sampling

$$Z \sim \mathcal{N}(0, I_K), \quad T' = \mathbf{1}_B \mu^T + Z L_\Sigma^T, \quad L_\Sigma L_\Sigma^T = \Sigma + \varepsilon I,$$

where L_Σ is the Cholesky factor of the (regularized) covariance.³

(b) *Weight sampling.* For weights modeled as $W \sim \mathcal{MN}(M, U, V)$, we sample

$$Z \sim \mathcal{N}(0, I_{M \times N}), \quad W' = M + L_U Z L_V^T, \\ L_U L_U^T = U + \varepsilon I, \quad L_V L_V^T = V + \varepsilon I.$$

This procedure preserves both row-wise and column-wise second-order correlations, yielding realistic surrogate weights and activations that match the training-time statistics without storing full tensors.

2) *Offline Error Quantification:* Using the learned distributions, LoKA Probe generates statistically representative test cases and compares low-precision kernel outputs and performance against high-precision references. This reveals layer-specific vulnerabilities that standard random testing misses, and helps us prune layers that do not benefit from low precision acceleration.

For each linear layer, we compute MERE and speedup using inputs and weights sampled from learned distributions rather than synthetic ones. Layers with high MERE scores or low speedups are flagged as problematic for low-precision execution. Figure 5 summarizes this workflow.

3) *LoKA Probe Key Findings:* We use LoKA Probe to analyze the Wukong model used in §II-C, sampling 100 realistic input-weight pairs for each linear layer and identified three critical patterns that result in large losses and unoptimal performance:

- **Problematic bias terms** Figure 6 shows different modes of L2 bias norm evolution during training. A significant portion of biases never converge, with some reaching values ≥ 0.1 . These diverging biases cascade through subsequent modules, causing out-of-bounds errors. When clamped and quantized, they can cause smaller values to vanish entirely.
- **Normalization overhead and errors** LayerNorm [10] operations require complex variance computations prone to mean cancellation when values are similar. This forces normalization to run in higher precision, requiring expensive

³We add a small jitter εI (e.g., $10^{-6} \times \frac{\text{trace}(\Sigma)}{K}$) to maintain numerical stability.

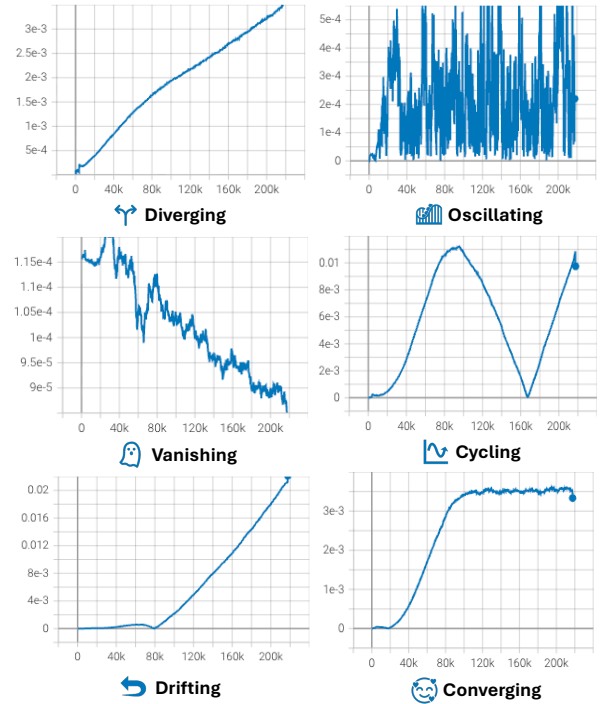


Fig. 6. Typical behaviors of bias norm of in Wukong training. Biases can introduce instability during training, especially under low-precision conditions.

dequantization before LayerNorm and requantization afterward, which is an overhead that often exceeds low-precision benefits, and prevents low-precision execution of such layers. Modern LRMs employ LayerNorms aggressively in between layers in MLP for training stability, creating a sizable impact on the end-to-end latency.

- **Sigmoid-based activation instability** LRM’s heavy use of sigmoid functions (Swish activation [62]: $x \cdot \sigma(x)$, SwishNorm: $x \cdot \sigma(\text{Norm}(x))$) involves exponential operations that amplify large elements while diminishing small ones, dramatically increasing quantization loss.

B. LoKA Mods: Precision-Aware Model-Hardware Co-design

The vulnerabilities identified by LoKA Probe (problematic bias terms, normalization overhead, and sigmoid instability) cannot be fixed by tightening kernel precision alone. Instead, we take a model-kernel co-design approach, introducing LoKA Mods: redesigned building blocks that improve both numerical stability and execution efficiency under low-precision conditions.

1) *No Bias:* We draw inspiration from recent LLM architectures that have moved away from bias terms. Models like DeepSeek eliminate biases from all feedforward and normalization layers, while PaLM [4] and Falcon [2] remove them from feedforward layers while retaining them in normalization components. Following this trend, we remove all bias terms from Wukong modules except the final prediction layers, where bias terms can be beneficial for different prediction tasks. This modification also provides the benefit of potentially reduced communication overheads: with FSDP per-parameter

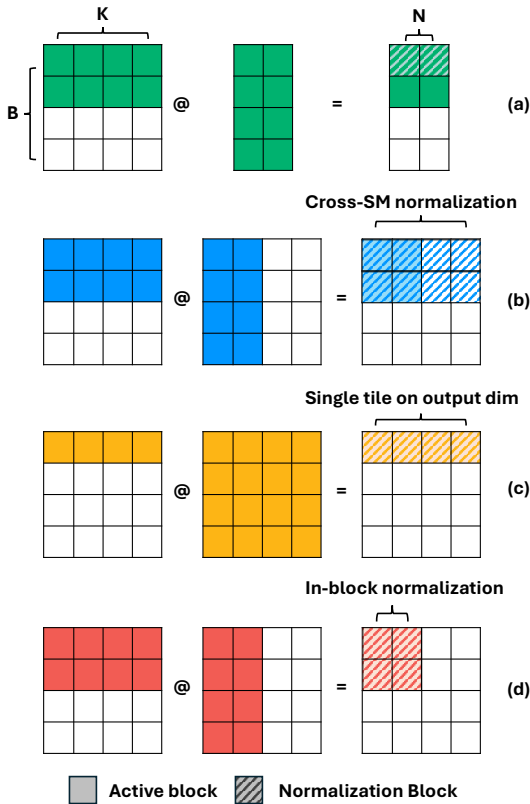


Fig. 7. BlockNorm design

padding [44], bias tensors smaller than the world size can incur significant overheads.

2) *Block-wise Normalization:* Our objective is to fuse normalization directly into the GEMM epilogue to minimize HBM I/O. While similar to epilogue fusion [65], our application is in a different context. By performing normalization immediately after GEMM completion while the output tiles still reside in on-chip memory (L1/L2 caches or registers), we avoid costly global memory traffic. However, conventional normalization layers operate along the feature dimension, which often misaligns with the physical data layout of GEMM outputs.

To address this, we propose **BlockNorm**, a normalization approach inspired by GroupNorm [71] which is originally designed to apply normalization within individual channels of a feature map. In our version of BlockNorm, instead of computing statistics across the full output dimension or individual feature, it applies *root-mean-square* normalization [77] across predefined blocks along the feature dimension, which happens within each computational block of the GEMM kernel:

$$\text{RMSNorm}((Wx + b).\text{view}(-1, \text{BlockN})).\text{view}(B, N).$$

We adopt RMSNorm over LayerNorm because it only normalizes by the L2-norm of activations, avoiding mean subtraction and thus reducing catastrophic cancellation errors under low-precision execution.

Case 1: Large batch, small output dimension. When the batch size B is large and the output dimension N is small

(Figure 7a), the entire feature vector required for normalization fits within a single thread block. In this regime, BlockNorm behaves identically to standard normalization layers, as all statistics are computed locally. Additional operations such as activation and (de)quantization can also be fused together before the final output is written to HBM.

Case 2: Small batch, large output dimension. When N is large, however, a single thread block can no longer hold an entire output row in shared memory. Computing RMS statistics now requires cross-block synchronization (Figure 7b), which negates most of the performance gains of fusion. One possible mitigation is to reduce the tile size along the batch dimension, but this incurs *SM wave quantization effects* [1] and lowers the L2 cache hit rate for the W matrix, yielding negligible end-to-end speedup (Figure 7c).

Tradeoff and design choice. We further explored semi-fused approaches where GEMM outputs partial statistics for later normalization, or tiling assignments aligned with the normalization axis at the SM level. While these approaches offer limited gains, they require extensive manual tuning for each new shape, which reduces generality. To achieve robust performance across diverse tensor shapes, LoKA deliberately relaxes full mathematical equivalence in favor of efficiency, leading to the final *BlockNorm* formulation (Figure 7d).

Concretely, BlockNorm normalizes over fixed-size blocks (e.g., 256 elements) rather than the full output dimension. During both training and inference, the same block size is used, and normalization statistics are computed independently per block to ensure train-test consistency.

Striking a Better Numerical and Performance Tradeoff with Model Co-design While the actual computation differs between BlockNorm and standard normalization practices, we argue that it does not break RMSNorm: it is mathematically equivalent to an unparameterized Grouped RMSNorm [71]. Global RMSNorm couples all channels to a single statistic, meaning one outlier suppresses all features. BlockNorm decouples feature subspaces, preventing catastrophic cancellation (which can prove important [39], [72]) and increasing the model’s representational capacity via more independent degrees of freedom. Because block size is strictly consistent between training and inference, the model natively adapts to this grouped topology without requiring global, hardware-inefficient synchronization. Empirically, BlockNorm improves stability over baselines (Figure 13), and convergence is insensitive to block size provided it is sufficiently large (e.g. 256) and identical across train/test phases. This approach also aligns closely with emerging Microscaling (MX) hardware standards, which utilize block-shared scaling to preserve dynamic range without the overhead of global synchronization [63]. The idea is also supported by prior work in the machine learning community. For example, pRMSNorm was proposed alongside RMSNorm [77] which assumes the identical distribution of the neurons and estimates RMS using as little as 6.25% of them. GroupNorm also found that normalizing over groups of output neurons remains effective for training stability and model quality. To demonstrate this, we compare training of

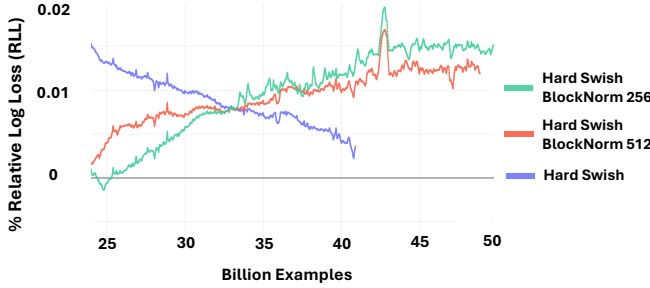


Fig. 8. Hard Swish and BlockNorm with sufficiently large block size converges to minimal loss.

a production Wukong model using a BlockNorm of size 256 with RMSNorm in Figure 8, demonstrate its ability to preserve model quality.

3) *Hard Swish*: Sigmoid-based swish activation and normalization come with large overheads and numerical challenges due to the expensive exponential operations [29], making Swish significantly slower than lightweight activations such as ReLU.

We replace normal swish function with Hard Swish [61], a piecewise linear, close approximation to Swish that significantly reduces computational cost while maintaining similar representational power. The Hard Swish function is defined as follows:

$$\text{h-swish}(x) = x \cdot \frac{\text{ReLU6}(x + 3)}{6}$$

where ReLU6 provides a bounded ReLU operation that outputs zero for negative inputs,

The elimination of exponential operations reduces computational overhead, while the piecewise linear nature of Hard Swish makes it naturally amenable to low-precision computation. Additionally, Hard Swish integrates seamlessly with BlockNorm, allowing both operations to be fused within the same kernel for maximum efficiency.

The representational power of Hard Swish remains comparable to standard Swish for the typical input ranges encountered in recommendation models, while providing substantially better behavior under limited numerical range. This trade-off of slightly simplified activation dynamics in exchange for dramatically improved low-precision stability proves highly beneficial in practice.

C. LoKA Dispatch: Per-Operator Kernel Orchestration

With improved low-precision accuracy enabled by LoKA Probe’s targeted analysis and LoKA Mods’ architectural improvements, the final challenge is achieving optimal end-to-end throughput. Even with numerically stable components, the diversity of GEMM shapes, hardware characteristics, and available libraries means that no single low-precision implementation performs best across all scenarios.

Modern low-precision ecosystems present a complex optimization landscape. Libraries differ fundamentally in their design philosophy: some provide bare kernels with maximum agility (e.g., DeepGEMM), others offer comprehensive quantization pipelines (e.g., FBGEMM), while still others function

as drop-in replacements for standard layers (TorchAO). Within each library, multiple recipes provide fine-grained control over forward and backward pass datatypes, quantization strategies (tensorwise, rowwise, blockwise), and other features like fast accumulation.

This diversity, while providing flexibility, creates a challenging optimization problem. Selecting the optimal combination of library and recipe for each GEMM operation requires balancing accuracy constraints with performance objectives, a task that becomes intractable to perform manually across hundreds of modules in production models.

Library and Recipe Selection LoKA Dispatch addresses this challenge through a lightweight runtime system that leverages the statistical insights from LoKA Probe to make optimal kernel selection decisions. Rather than applying uniform low-precision policies across all operations, LoKA Dispatch treats each GEMM as an independent optimization opportunity, selecting the fastest implementation that satisfies accuracy requirements.

The core algorithm operates through constrained optimization. For each GEMM operation, candidate implementations are first filtered by accuracy constraints derived from LoKA Probe’s MERE analysis. Only libraries and recipes whose expected error falls below a conservative threshold (typically $\text{MERE} < 0.2$) and whose speedup (per LoKA Probe analysis) exceeds a minimum improvement factor (typically $> 1.05\times$) are considered. From this filtered set, LoKA Dispatch selects the implementation with the highest measured throughput. The heterogeneous nature of this mapping underscores the importance of fine-grained, per-operation optimization rather than global policies.

Trainer Integration The technical implementation of LoKA Dispatch centers on providing a unified interface across diverse low-precision libraries while maintaining compatibility with existing training frameworks. We implement a custom autograd function that serves as a universal adapter, allowing the same high-level interface to route to different underlying kernel implementations.

During model initialization, we perform a transformation pass that identifies target linear layers and replaces them with LoKA-aware linear wrappers. These wrappers encapsulate the dispatch logic while maintaining identical semantics to standard PyTorch linear layers, ensuring compatibility with existing training pipelines and optimization strategies. The autograd function called by the wrappers handles both forward and backward pass routing, maintaining separate optimization decisions for each direction when beneficial. This granular control proves particularly valuable because forward and backward passes often have different optimal implementations due to varying tensor shapes, layouts, datatypes and computational patterns.

It’s worth noting that LoKA Dispatch does not need to operate dynamically per-iteration once static profiling results are obtained. For inference, kernel selection is entirely statically determined, and distribution shifts are handled naturally via on-line continuous training. For training, Dispatch’s dynamism acts purely as a safeguard against massive distribution shifts (e.g., sudden user behavior changes during a holiday). Dynamically

switching precision mid-training does incur a recompilation tax and is often not worthwhile.

IV. EVALUATION

We comprehensively evaluate LoKA across representative LRM architectures in production settings, demonstrating its effectiveness in enabling stable low-precision training and inference while delivering substantial throughput improvements. Our evaluation addresses three key questions: Can LoKA achieve lossless low-precision training? What end-to-end performance gains does it deliver across different scales and hardware? How do individual LoKA components contribute to overall effectiveness?

A. Setup

Our evaluation focuses on three families of state-of-the-art LRM architectures that represent different points in the design space: Wukong [78], InterFormer [74] and External Large Foundation Model [43]. This selection provides a complete coverage of industry-scale recommenders that also assess open-source components (e.g., Wukong’s FMB and LCB [5], InterFormer’s Transformer [67] components, and integration of DCN [70], DHEN [79], DLRM [53] and SUM [80] architecture in ELFM). All experiments are conducted on industry-scale datasets that contains tens of billions of examples, each with thousands of features, following the practice in [78].

We conduct experiments across diverse hardware configurations (Nvidia H100/B200 and AMD MI300X clusters with 16-256 GPUs) to capture performance characteristics across different generations and scales. Our implementation integrates LoKA with the PyTorch framework and supports three major low-precision libraries: TorchAO, DeepGEMM, and FBGEMM. All experiments focus on FP8 training and inference, representing the most practical current low-precision target for production deployment. We use TorchRec [37], hybrid parallelism [50] with balanced sharding [75] and FSDP [81] for training to minimize communication overheads. We also enable quantized communication [73] for all models in BF16 format regardless of compute datatype to ensure fair comparisons across all experiments.

We evaluate LoKA across three model configurations and model families representing different scales and computational characteristics in Table IV.

Model Family	GFLOPs/sample	Params (B)	Batch Size & GPU Count		
			H100	B200/MI 300X	GB200/MI 350X
Wukong	24	257	6K&32	12K&16	20K&16
Interformer	28	566	4K&64	8K&64	20K&32
ELFM	40	1343	2k&256	6K&128	20K&32

TABLE IV
MODEL SPECIFICATIONS AND TRAINING SETUP

The experimental methodology emphasizes production realism. We use actual training data (our dataset contains thousands of features), realistic batch sizes, and production-scale model configurations. This approach ensures that our results reflect the performance characteristics that would be observed in real world scenarios.

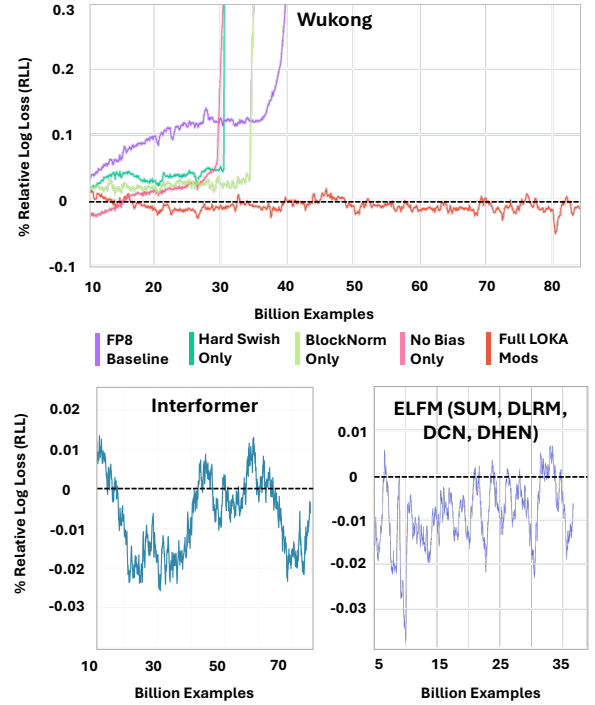


Fig. 9. Lossless full-trajectory FP8 training of Wukong, Interformer and ELFM with LoKA.

B. Lossless FP8 LRM Training

The fundamental test of LoKA effectiveness is whether it can enable stable FP8 training without accuracy degradation. Figure 9 demonstrates this capability on the three models, compared to original, unmodified baselines. We plotted the relative log loss curve with respect to the baseline, excluding model warmup time and we train enough examples until the relative log loss curve converges. Evidently, LoKA was not only able to achieve quality neutrality at the end, it was able to do so during the full training trajectory, meaning that within each sampling window, LoKA has delivered on par quality consistently. This is especially important for LRMs that process streaming data with shifting distributions. Notably, Figure 9 (top) repeated the same model setup, and the dramatic FP8 training failure in Figure 3 has succeeded with LoKA.

Ablation To quantify the contribution to training stability from each LoKA Mod, we quantify its effect in Figure 9 (top): no Bias reduces early training instability, BlockNorm provides better numerical conditioning, and Hard Swish reduces activation-related errors. However, none of these modifications alone achieves full stability. Instead, when all LoKA Mods are combined, the full system achieves complete loss neutrality throughout training, matching the accuracy of high-precision baselines while operating in FP8.

C. End-to-end Speedup

Figure 10 shows the end-to-end training and (in-trainer) inference speedup results, over strong baselines (i.e., models that have been rewritten with LoKA Mods), revealing several important patterns. LoKA achieves speedups up to $1.19\times$ and

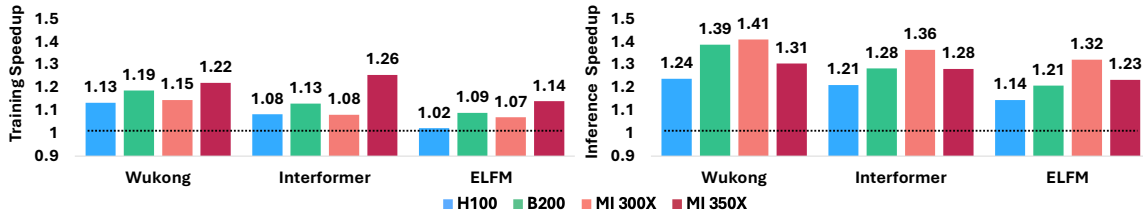


Fig. 10. End-to-end Speedup of LoKA Training (Left) and Inference (Right)

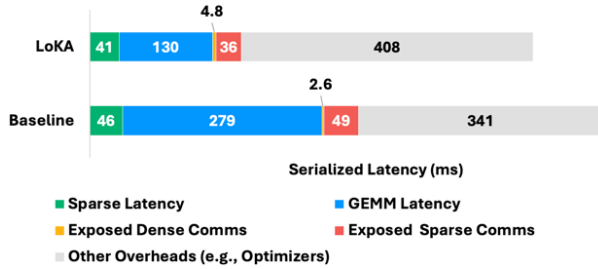


Fig. 11. End-to-end latency breakdown of training with and without LoKA

1.4 $\times$ respectively in training and inference, with performance gains varying systematically based on model and hardware characteristics.

LoKA’s effectiveness correlates strongly with the compute-to-communication ratio. Models trained at smaller scales with higher computational intensity benefit more from low-precision acceleration, while larger-scale training with more communication overhead sees reduced benefits.

More recent GPU architectures (B200 vs H100) show larger LoKA speedups, primarily due to increased memory capacity enabling larger batch sizes which improves the cost-benefit ratio of low-precision computation.

All tested model architectures benefit substantially from LoKA, demonstrating that the framework generalizes across different LRM design approaches rather than being specific to particular architectural choices.

Ablation. To quantify LoKA’s contribution to end-to-end training latency, we profile the iteration-level runtime breakdown before and after applying LoKA, using iteration 706 (a representative iteration with stable throughput) averaged across all ranks. Figure 11 reports the component-wise latency for the Wukong model trained on 16 B200 GPUs. The majority of the observed speedup originates from reduced GEMM latency (approximately 2 $\times$). Since LoKA does not modify the sparse embedding pipeline or inter-GPU communication pattern, the slight difference in sparse communication time is attributed to normal run-to-run variability rather than systematic change.

Scalability. We further evaluate end-to-end throughput scalability using the same Wukong model on 16–256 GPUs. As shown in Figure 12, LoKA maintains substantial performance gains at scale, achieving a 10% throughput improvement at 256 GPUs despite increased communication overhead. While the relative gain diminishes with larger cluster sizes, primarily due to the rising proportion of synchronization and embedding

	Faulty Test Code	Fixed Test Code
Normal Distribution	0.42	0.42
LoKA Probe	17.04	0.37

TABLE V

NUMERICAL ERROR DETECTION VIA LoKA PROBE’S MERE.

communication [14], [46]–[48], the results confirm LoKA’s effectiveness and practicality in large-scale distributed training.

D. Component Analysis

We conduct detailed component-level studies examining each LoKA component’s impact on both accuracy and performance on representative benchmarks.

1) *LoKA Probe*: We validate LoKA Probe’s core value of higher sensitivity in detecting numerical errors through realistic distribution modeling by expanding the MERE measurements study in Table II to cover all 27 linear layers from the subject model and summarize the overall results in Table V.

Coincidentally, with the help of LoKA Probe, we were also able to discover a faulty implementation in the FBGEMM library’s production benchmark: when generating tests with random inputs, the MEREs from correct and incorrect implementation are almost identical, but when using LoKA Probe-generated inputs, the MEREs differed by 47 $\times$, which prompted us to investigate and fix with its developers.

2) *LoKA Mods*: Figure 13 quantifies the individual and combined effects of LoKA Mods on computational latency using representative GEMM shapes with batch size 1024 (typical choice), with torch compile [6] enabled. The analysis separates linear layer latency from activation and normalization overhead to isolate the impact of each modification. No Bias provides the largest single contribution to latency reduction, eliminating parameter overhead and simplifying computation paths. BlockNorm delivers substantial improvements by enabling fusion optimizations impossible with standard LayerNorm. Hard Swish contributes meaningful but smaller gains through elimination of expensive exponential operations. The combined effect achieves over 2 $\times$ latency reduction.

3) *LoKA Dispatch*: To isolate LoKA Dispatch’s contribution, we compare it against uniform application of single-library recipes using the same Wukong setup in compute-only mode (eliminating communication effects). The baseline approaches apply consistent recipes across all GEMM operations: TorchAO with tensorwise (TW), rowwise (RW), or a mixed recipe that uses rowwise scaling in the forward but high precision for input gradient and tensorwise scaling for weight in the backward

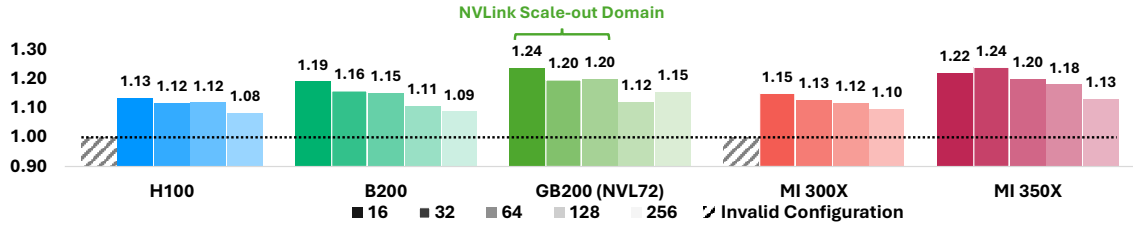


Fig. 12. Scalability of LoKA on Wukong training, varying number of GPUs. N/A: configuration invalid.

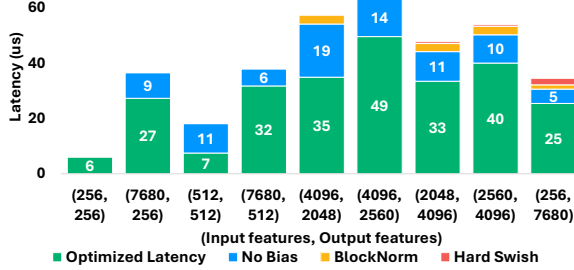


Fig. 13. Assessing LoKA Mods' effectiveness on reducing latency of common GEMM sizes, by ablating the latency reduction of each component.

(RW GW HP); DeepGEMM with blockwise (BW) scaling; and FBGEMM with rowwise scaling.

As shown in Table VI, LoKA Dispatch's $1.12\times$ speedup exceeds the best single-recipe approach ($1.08\times$), demonstrating the value of per-operation optimization. The performance advantage stems from matching each GEMM's characteristics to the most appropriate implementation rather than applying uniform policies that may be suboptimal for specific tensor shapes or computational patterns.

	TorchAO		DeepGEMM		FBGEMM	LoKA Dispatch
TW	RW	RW GW HP	BW	BW	RW	Mixed
1.05	1.01	1.08	0.85	0.98		1.12

TABLE VI
SPEEDUP OF LoKA DISPATCH

V. PRODUCTION DEPLOYMENTS

We have successfully deployed LoKA to the flagship models of production advertising recommendation services at a major social media company serving billions of users. LoKA has demonstrated 5-20% end-to-end training throughput and 10-17% speedup in production inference across various launches.

VI. FURTHER DISCUSSIONS

A. Related Work

Quantization. Early work on quantized neural networks demonstrated that models can be trained end-to-end with low-precision weights and activations [36]. Building on this foundation, a large body of research has explored quantization to improve model efficiency across training and inference. Existing methods span post-training quantization (PTQ) [11], [45], [72], quantization-aware training (QAT) [38], and mixed-precision techniques [49], each balancing accuracy, retraining

cost, and implementation complexity. Comprehensive surveys [32], [51] and adaptive schemes such as layer-wise or adaptive quantization [24], [26] further analyze these trade-offs. More recently, direct low-precision (DLP) training has emerged, pushing computation into low-bit formats such as FP8 and FP4. Examples include industrial-scale FP8 deployments [19] and native FP4 training frameworks [12], [16], [23]. While these efforts establish the feasibility of sub-8-bit arithmetic, they focus on homogeneous transformer or CNN workloads. In contrast, LoKA targets large-scale recommendation models whose small-GEMM kernels, normalization-heavy dataflow, and statistical heterogeneity violate the assumptions underlying existing quantization strategies.

Libraries. Several production-grade libraries implement low-precision inference and training across diverse hardware backends. FBGEMM [41] pioneered high-performance INT8 and FP16 kernels optimized for x86 and GPU architectures, forming the foundation of many industrial recommendation deployments. DeepGEMM [17] and TorchAO [60] extend this direction to FP8 training, providing fine-grained scaling recipes at both CUDA and PyTorch-level. Other frameworks such as NVIDIA's Transformer Engine [55] and AMD Quark [3] expose similar quantization abstractions.

While these systems significantly advance kernel-level efficiency, they rely on synthetic benchmarks and generic GEMM shapes for accuracy validation, which often diverge from real LRM distributions. As shown in [32], kernel accuracy depends strongly on input statistics, a factor largely unmodeled in current implementations. Consequently, production LRMs continue to train predominantly in BF16 or TF32 precision, with FP8 adoption limited to post-training inference. LoKA complements these libraries by introducing a statistical probing and runtime selection layer (LoKA Probe) that characterizes per-layer distributions online and dispatches to the optimal low-precision kernel at runtime (LoKA Dispatch).

Generalizability. LoKA demonstrates strong generalizability across a wide spectrum of large recommendation model architectures and operators. Its techniques apply seamlessly to the factorization-machine and linear-compression modules in Wukong [78], the transformer-based sequence processors and interleaved non-sequential components in InterFormer [74], as well as the compound architectures used by ELFM [43], which integrate DHEN [79], SUM [80], DLRM [52], FMB [5] and DCN [70]. LoKA operates directly on the common computational primitives of LRMs—linear transformations, normalization, and activation functions—making its methods

agnostic to architectural topology. This property allows LoKA to extend naturally to emerging hybrid or foundation-scale recommendation models that combine sequential, graph-based, and multimodal components.

A key test of LoKA’s principles is whether they transfer to hardware unseen during framework development. LoKA was designed and validated on H100, B200, and MI300X clusters. Subsequently, without any modification to LoKA’s methodology or components, we evaluated on GB200 NVL72 and MI350X — hardware that was not available during LoKA’s development. As shown in Figures 10 and 12, LoKA delivered comparable speedups on both new platforms, confirming that its principles are not overfit to specific hardware characteristics.

B. Limitations

LoKA Probe’s benchmarks cannot fully capture real-world performance due to compute-communication overlap, though SM carveout [54] techniques help mitigate this. The framework requires models to use standard building blocks, as novel architectures may need additional kernel development. LoKA Dispatch may require manual intervention when introducing new low-precision kernels to ensure proper integration with compilation frameworks like torch.compile [6] for best performance. Additionally, error-based probing [45] including LoKA lacks a mechanism to reason about error propagation throughout the network, making it more conservative than needed: for example, even large errors at each operator may cancel out at the end leading to neutral quality, but LoKA Dispatch will disable low precision for all such layers, leading to potential missed opportunities.

C. Future Work

We focus on FP8 as the most practical current target for production deployment. While FP4 and other ultra-low precisions show promise, they remain active research areas for dense model training [12], [16], [23], [25], [36], [56], [69]. Recent study shows Random Hadamard Transform [8], [9], [20], [42] and quantized communication [28] can further improve LoKA efficacy. Orthogonally, AutoML [21], [68] may aid in bridging the gap between MERE and final accuracy.

VII. CONCLUSION

Applying low-precision training to LLMs faces challenges from numerical instability and suboptimal throughput gains. We propose LoKA, a systematic framework built on three generalizable principles: (1) distribution-aware profiling to identify where low precision is safe, (2) precision-aware model-hardware co-design to expand where it is safe, and (3) per-operator kernel orchestration to maximize the resulting gains. Our evaluation across three model families and five GPU architectures demonstrates up to 20% training throughput improvements and 40% inference acceleration with no quality loss. LoKA has been deployed to flagship models at a major social media company, serving billions of users.

REFERENCES

- [1] “Matrix multiplication background user’s guide - nvidia docs,” [Online; accessed 2025-09-11]. [Online]. Available: <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#wave-quant>
- [2] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, Étienne Goffinet, D. Hesslow, J. Launay, Q. Malartic, D. Mazzotta, B. Noune, B. Pannier, and G. Penedo, “The falcon series of open language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2311.16867>
- [3] AMD, “Introduction — amd quark 0.10 documentation,” [Online; accessed 2025-10-29]. [Online]. Available: https://quark.docs.amd.com/latest/onnx/tutorial_microexponents_quantization.html
- [4] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu, “Palm 2 technical report,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.10403>
- [5] Anonymous, “Dot product matrix compression for machine learning,” *Technical Disclosure Commons*, 2019.
- [6] J. Ansel, E. Yang, H. He, N. Gimselshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 929–947. [Online]. Available: <https://doi.org/10.1145/3620665.3640366>
- [7] N. Ardalani, C.-J. Wu, Z. Chen, B. Bhushanam, and A. Aziz, “Understanding scaling laws for recommendation models,” *arXiv preprint arXiv:2208.08489*, 2022.
- [8] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, “Quarot: Outlier-free 4-bit inference in rotated llms,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 213–100 240, 2024.
- [9] S. Ashkboos, M. Nikdan, S. Tabesh, R. L. Castro, T. Hoefler, and D. Alistarh, “Halo: Hadamard-assisted lower-precision optimization for llms,” *arXiv preprint arXiv:2501.02625*, 2025.
- [10] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>
- [11] R. Banner, Y. Nahshan, E. Hoffer, and D. Soudry, “Post-training 4-bit quantization of convolution networks for rapid-deployment,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.05723>
- [12] R. L. Castro, A. Panferov, S. Tabesh, O. Sieberling, J. Chen, M. Nikdan, S. Ashkboos, and D. Alistarh, “Quartet: Native fp4 training can be optimal for large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.14669>
- [13] T. F. Chan, G. H. Golub, and R. J. LeVeque, “Algorithms for computing the sample variance: Analysis and recommendations,” *The American Statistician*, vol. 37, no. 3, pp. 242–247, 1983.

- [14] J. Chen, H. Zhang, W. Zhang, L. Luo, J. Chase, I. Stoica, and D. Zhuo, "NetHint: White-Box networking for Multi-Tenant data centers," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. Renton, WA: USENIX Association, Apr. 2022, pp. 1327–1343. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/chen-jingrong>
- [15] W. Cheng, Y. Shen, and L. Huang, "Adaptive factorization network: Learning adaptive-order feature interactions," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 3609–3616.
- [16] B. Chmiel, M. Fishman, R. Banner, and D. Soudry, "Fp4 all the way: Fully quantized training of llms," 2025. [Online]. Available: <https://arxiv.org/abs/2505.19115>
- [17] DeepSeek, "Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling," [Online; accessed 2025-08-29]. [Online]. Available: <https://github.com/deepseek-ai/DeepGEMM>
- [18] DeepSeek-AI, "Deepgemm numerical test," https://github.com/deepseek-ai/DeepGEMM/blob/main/tests/test_bf16.py#L38, [Accessed 17-02-2026].
- [19] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [20] DeepSeek-AI, A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, C. Lu, C. Zhao, C. Deng, C. Xu, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, E. Li, F. Zhou, F. Lin, F. Dai, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Li, H. Liang, H. Wei, H. Zhang, H. Luo, H. Ji, H. Ding, H. Tang, H. Cao, H. Gao, H. Qu, H. Zeng, J. Huang, J. Li, J. Xu, J. Hu, J. Chen, J. Xiang, J. Yuan, J. Cheng, J. Zhu, J. Ran, J. Jiang, J. Qiu, J. Li, J. Song, K. Dong, K. Gao, K. Guan, K. Huang, K. Zhou, K. Huang, K. Yu, L. Wang, L. Zhang, L. Wang, L. Zhao, L. Yin, L. Guo, L. Luo, L. Ma, L. Wang, L. Zhang, M. S. Di, M. Y. Xu, M. Zhang, M. Zhang, M. Tang, M. Zhou, P. Huang, P. Cong, P. Wang, Q. Wang, Q. Zhu, Q. Li, Q. Chen, Q. Du, R. Xu, R. Ge, R. Zhang, R. Pan, R. Wang, R. Yin, R. Xu, R. Shen, R. Zhang, S. H. Liu, S. Lu, S. Zhou, S. Chen, S. Cai, S. Chen, S. Hu, S. Liu, S. Hu, S. Ma, S. Wang, S. Yu, S. Zhou, S. Pan, S. Zhou, T. Ni, T. Yun, T. Pei, T. Ye, T. Yue, W. Zeng, W. Liu, W. Liang, W. Pang, W. Luo, W. Gao, W. Zhang, X. Gao, X. Wang, X. Bi, X. Liu, X. Wang, X. Chen, X. Zhang, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Li, X. Yang, X. Li, X. Chen, X. Su, X. Pan, X. Lin, X. Fu, Y. Q. Wang, Y. Zhang, Y. Xu, Y. Ma, Y. Li, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Qian, Y. Yu, Y. Zhang, Y. Ding, Y. Shi, Y. Xiong, Y. He, Y. Zhou, Y. Zhong, Y. Piao, Y. Wang, Y. Chen, Y. Tan, Y. Wei, Y. Ma, Y. Liu, Y. Yang, Y. Guo, Y. Wu, Y. Wu, Y. Cheng, Y. Ou, Y. Xu, Y. Wang, Y. Gong, Y. Wu, Y. Zou, Y. Li, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Zhao, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Huang, Z. Wu, Z. Li, Z. Zhang, Z. Xu, Z. Wang, Z. Gu, Z. Zhu, Z. Li, Z. Zhang, Z. Xie, Z. Gao, Z. Pan, Z. Yao, B. Feng, H. Li, J. L. Cai, J. Ni, L. Xu, M. Li, N. Tian, R. J. Chen, R. L. Jin, S. S. Li, S. Zhou, T. Sun, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Song, X. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Z. Huang, Z. Xu, Z. Zhang, D. Ji, J. Liang, J. Guo, J. Chen, L. Xia, M. Wang, M. Li, P. Zhang, R. Chen, S. Sun, S. Wu, S. Ye, T. Wang, W. L. Xiao, W. An, X. Wang, X. Sun, X. Wang, Y. Tang, Y. Zha, Z. Zhang, Z. Ju, Z. Zhang, and Z. Qu, "Deepseek-v3.2: Pushing the frontier of open large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2512.02556>
- [21] K. Deng, H. Zheng, M. Qing, K. Zhu, G. Li, Y. Xiao, L. E. Zhang, L. Guo, B. Hui, Y. Wang, G. Yuan, G. Agrawal, W. Niu, and X. Ma, "From bits to chips: An llm-based hardware-aware quantization agent for streamlined deployment of llms," 2026. [Online]. Available: <https://arxiv.org/abs/2601.03484>
- [22] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," 2022. [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [23] K. Devleker, "Nvfp4 trains with precision of 16-bit and speed and efficiency of 4-bit — nvidia technical blog," 8 2025, [Online; accessed 2025-08-31]. [Online]. Available: <https://developer.nvidia.com/blog/nvfp4-trains-with-precision-of-16-bit-and-speed-and-efficiency-of-4-bit/>
- [24] R.-G. Dumitru, V. Yadav, R. Maheshwary, P.-I. Clotan, S. T. Madhusudhan, and M. Surdeanu, "Layer-wise quantization: A pragmatic and effective method for quantizing llms beyond integer bit-levels," 2024. [Online]. Available: <https://arxiv.org/abs/2406.17415>
- [25] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," 2020. [Online]. Available: <https://arxiv.org/abs/1902.08153>
- [26] F. Faghri, I. Tabrizian, I. Markov, D. Alistarh, D. Roy, and A. Ramezani-Kebrya, "Adaptive gradient quantization for data-parallel sg," 2020. [Online]. Available: <https://arxiv.org/abs/2010.12460>
- [27] FBGEMM, "Fbgemm numerical test," https://github.com/pytorch/FBGEMM/blob/main/fbgemm_gpu/test/quantize/fused_8bit_rowwise_test.py#L61rely, [Accessed 17-02-2026].
- [28] W. Feng, "Enabling float8 all-gather in fsdp2 - distributed - pytorch developer mailing list," 8 2024, [Online; accessed 2025-10-29]. [Online]. Available: <https://dev-discuss.pytorch.org/t/enabling-float8-all-gather-in-fsdp2/2359>
- [29] M. Fishman, B. Chmiel, R. Banner, and D. Soudry, "Scaling fp8 training to trillion-token llms," *arXiv preprint arXiv:2409.12517*, 2024.
- [30] X. Gao, S. Acharya, S. Han, Y. Ren, Y. Zhao, L. Luo, C. Wang, P. Fernando, S. Mishra, S. Yan *et al.*, "Deck: Experiences on delta checkpointing for industrial recommendation systems," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4978–4990, 2025.
- [31] S. Geng, J. Tan, S. Liu, Z. Fu, and Y. Zhang, "Vip5: Towards multimodal foundation models for recommendation," *arXiv preprint arXiv:2305.14302*, 2023.
- [32] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," 2021. [Online]. Available: <https://arxiv.org/abs/2103.13630>
- [33] X. Guo, J. Pan, X. Wang, B. Chen, J. Jiang, and M. Long, "On the embedding collapse when scaling up recommendation models," *arXiv preprint arXiv:2310.04400*, 2023.
- [34] D. A. Harville, "Matrix algebra from a statistician's perspective," 1998.
- [35] A. Hernández-Cano, D. Garbaya, I. Schlag, and M. Jaggi, "Towards fully fp8 gemm llm training at scale," *arXiv preprint arXiv:2505.20524*, 2025.
- [36] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks: Training neural networks with low precision weights and activations," 2016. [Online]. Available: <https://arxiv.org/abs/1609.07061>
- [37] D. Ivchenko, D. Van Der Staay, C. Taylor, X. Liu, W. Feng, R. Kindi, A. Sudarshan, and S. Sefati, "Torchrec: a pytorch domain library for recommendation systems," in *Proceedings of the 16th ACM Conference on Recommender Systems*, 2022, pp. 482–483.
- [38] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," 2017. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [39] M. Jin, K. Mei, W. Xu, M. Sun, R. Tang, M. Du, Z. Liu, and Y. Zhang, "Massive values in self-attention modules are the key to contextual knowledge understanding," 2025. [Online]. Available: <https://arxiv.org/abs/2502.01563>
- [40] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

- [41] D. Khudia, J. Huang, P. Basu, S. Deng, H. Liu, J. Park, and M. Smelyanskiy, “Fbgemm: Enabling high-performance low-precision deep learning inference,” 2021. [Online]. Available: <https://arxiv.org/abs/2101.05615>
- [42] H. A. D. Le, S. Joshi, Z. Yang, Z. Xu, and A. Shrivastava, “Scout before you attend: Sketch-and-walk sparse attention for efficient llm inference,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.07397>
- [43] M. Liang, X. Liu, R. Jin, B. Liu, Q. Suo, Q. Zhou, S. Zhou, L. Chen, H. Zheng, Z. Li, S. Jiang, J. Yang, X. Xia, F. Yang, Y. Badr, E. Wen, S. Xu, H. Chen, Z. Zhang, J. Nie, C. Yang, Z. Zeng, W. Zhang, X. Huang, Q. Li, S. Wang, E. Lyu, W. Lu, R. Zhang, W. Wang, J. Rudy, M. Hang, K. Wang, Y. Ma, S. Wang, S. Zeng, T. Tang, X. Wei, L. Jin, J. Zhang, M. Chen, J. Xu, A. Huang, X. Zeng, C. Zhang, Z. Zhao, J. Yang, Q. Jin, X. Chen, A. A. Amlesahwaram, L. Song, L. Luo, Y. Hao, N. Xiao, Y. Yetim, L. Pan, G. Liu, Y. Hu, Y. Huang, J. Xu, R. Zhu, X. Zhang, Y. Liu, H. Yin, Y. Chen, B. Zhang, X. Liu, X. Wang, W. Mao, Z. Li, Z. Zhou, F. Gu, Q. Huang, C. Sun, N. Yu, S. Gu, S. Mao, B. Au, J. Qin, P. Yao, J.-W. Choi, B. Gao, E. Wang, L. Zhang, W.-Y. Chen, T. Lee, Y. Zha, Y. Meng, A. Gong, E. Gao, J. Hsueh, J. Zheng, A. Vahdipour, Y. Han, Y. Yao, T. Kureha, S. Chang, M. Sultan, J. Bocharov, S. Chordia, X. Gan, P. Sun, R. Liu, B. Long, W. Chen, S. Kolay, and H. Li, “External large foundation model: How to efficiently serve trillions of parameters for online ads recommendation,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.17494>
- [44] W. Liang, T. Liu, L. Wright, W. Constable, A. Gu, C.-C. Huang, I. Zhang, W. Feng, H. Huang, J. Wang, S. Purandare, G. Nadathur, and S. Idreos, “TorchTitan: One-stop pytorch native solution for production ready llm pre-training,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.06511>
- [45] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 87–100. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf
- [46] L. Luo, J. Nelson, L. Ceze, A. Phanishayee, and A. Krishnamurthy, “Parameter hub: a rack-scale parameter server for distributed deep neural network training,” in *Proceedings of the ACM Symposium on Cloud Computing*, 2018, pp. 41–54.
- [47] L. Luo, P. West, J. Nelson, A. Krishnamurthy, and L. Ceze, “Plink: Discovering and exploiting locality for accelerated distributed training on the public cloud,” *Proceedings of Machine Learning and Systems*, vol. 2, pp. 82–97, 2020.
- [48] L. Luo, B. Zhang, M. Tsang, Y. Ma, C.-H. Chu, Y. Chen, S. Li, Y. Hao, Y. Zhao, G. Lakshminarayanan *et al.*, “Disaggregated multi-tower: Topology-aware modeling technique for efficient large-scale recommendation,” *arXiv preprint arXiv:2403.00877*, 2024.
- [49] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.03740>
- [50] D. Mudigere, Y. Hao, J. Huang, A. Tulloch, S. Sridharan, X. Liu, M. Ozdal, J. Nie, J. Park, L. Luo *et al.*, “High-performance, distributed training of large-scale deep learning recommendation models,” *arXiv preprint arXiv:2104.05158*, 2021.
- [51] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.08295>
- [52] M. Naumov, D. Mudigere, H.-J. M. Shi, J. Huang, N. Sundaraman, J. Park, X. Wang, U. Gupta, C.-J. Wu, A. G. Azzolini, D. Dzhulgakov, A. Malleevich, I. Cherniavskii, Y. Lu, R. Krishnamoorthi, A. Yu, V. Kondratenko, S. Pereira, X. Chen, W. Chen, V. Rao, B. Jia, L. Xiong, and M. Smelyanskiy, “Deep learning recommendation model for personalization and recommendation systems,” 2019. [Online]. Available: <https://arxiv.org/abs/1906.00091>
- [53] M. Naumov, D. Mudigere, H.-J. M. Shi, J. Huang, N. Sundaraman, J. Park, X. Wang, U. Gupta, C.-J. Wu, A. G. Azzolini *et al.*, “Deep learning recommendation model for personalization and recommendation systems,” *arXiv preprint arXiv:1906.00091*, 2019.
- [54] Nvidia, “1. nvidia ampere gpu architecture tuning guide — ampere tuning guide 13.0 documentation,” [Online; accessed 2025-09-12]. [Online]. Available: <https://docs.nvidia.com/cuda/ampere-tuning-guide/index.html>
- [55] NVIDIA, “Github - nvidia/transformerengine: A library for accelerating transformer models on nvidia gpus, including using 8-bit floating point (fp8) precision on hopper, ada and blackwell gpus, to provide better performance with lower memory utilization in both training and inference.” [Online; accessed 2025-10-29]. [Online]. Available: <https://github.com/NVIDIA/TransformerEngine>
- [56] NVIDIA, F. Abecassis, A. Agrusa, D. Ahn, J. Alben, S. Alborghetti, M. Andersch, S. Arayandi, A. Bjorlin, A. Blakeman, E. Briones, I. Buck, B. Catanzaro, J. Choi, M. Chrzanowski, E. Chung, V. Cui, S. Dai, B. D. Rouhani, C. del Mundo, D. Donia, B. Eryilmaz, H. Estela, A. Goel, O. Goncharov, Y. Guvvala, R. Hesse, R. Hewett, H. Hum, U. Kapasi, B. Khailany, M. Khona, N. Knight, A. Kondratenko, R. Krashinsky, B. Lanir, S. Layton, M. Lightstone, D. Lo, P. Micikevicius, A. Mishra, T. Moon, D. Narayanan, C. Ni, A. Paithankar, S. Pasumarthi, A. Patel, M. Patwary, A. Poojary, G. Prasad, S. Priyadarshi, Y. Qin, X. Ren, O. Rybakov, C. Sakr, S. Satheesh, S. Sergienko, P. Shamis, K. Shankar, N. Sharma, M. Shoeby, M. Siu, M. Smelyanskiy, D. Stolic, D. Stolic, B.-Y. Su, F. Sun, N. Tajbakhsh, S. Thomas, P. Treadak, E. Tsykunov, G. Vaithilingam, A. Vavre, R. Venkatesan, R. Waleffe, Q. Wan, H. Wang, M. Wang, L. Wei, H. Wu, E. Wu, K. Wyss, N. Xu, J. Xue, C. Yang, Y. Zhai, R. Zhang, J. Zhu, and Z. Zhu, “Pretraining large language models with nvfp4,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.25149>
- [57] NVIDIA Corporation, “Nvidia a100 gpu datasheet,” [Online; accessed 2025-08-25]. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet.pdf>
- [58] —, “Nvidia b200 gpu datasheet,” [Online; accessed 2025-08-25]. [Online]. Available: <https://nvdam.widen.net/s/wwnsxrh2w/blackwell-datasheet-3384703>
- [59] —, “Nvidia h100 gpu datasheet,” [Online; accessed 2025-08-25]. [Online]. Available: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-tensor-core-gpu-datasheet?ncid=no-ncid>
- [60] A. Or, A. Jain, D. Vega-Myhre, J. Cai, C. D. Hernandez, Z. Zheng, D. Guessous, V. Kuznetsov, C. Puhres, M. Saroufim, S. Rao, T. Tran, and A. Samardžić, “Torchao: Pytorch-native training-to-serving model optimization,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.16099>
- [61] S. A. Pydimarthy, S. M. Khairnar, S. G. Palacios, G. Sankaranarayanan, D. Hoagland, D. Nepomnashchy, and H. P. Nguyen, “Evaluating model performance with hard-swish activation function adjustments,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.06879>
- [62] P. Ramachandran, B. Zoph, and Q. V. Le, “Swish: a self-gated activation function,” *arXiv: Neural and Evolutionary Computing*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:196158220>
- [63] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung, “Microscaling data formats for deep learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.10537>
- [64] J. Tang, Y. Drori, D. Chang, M. Sathiamoorthy, J. Gilmer, L. Wei, X. Yi, L. Hong, and E. H. Chi, “Improving training stability for multitask ranking models in recommender systems,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 4882–4893. [Online]. Available: <https://doi.org/10.1145/3580305.3599846>
- [65] P. Tillet, H. T. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, ser. MAPL 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 10–19. [Online]. Available: <https://doi.org/10.1145/3315508.3329973>
- [66] TorchAO, “Torchao blockwise triton test,” https://github.com/pytorch/ao/blob/main/test/kernel/test_blockwise_triton.py#L55, [Accessed 17-02-2026].
- [67] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [68] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, “Hq: Hardware-aware automated quantization with mixed precision,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 8612–8620.
- [69] R. Wang, Y. Gong, X. Liu, G. Zhao, Z. Yang, B. Guo, Z. Zha,

- and P. Cheng, “Optimizing large language model training using fp4 quantization,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.17116>
- [70] R. Wang, R. Shivanna, D. Cheng, S. Jain, D. Lin, L. Hong, and E. Chi, “Den v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems,” in *Proceedings of the web conference 2021*, 2021, pp. 1785–1797.
- [71] Y. Wu and K. He, “Group normalization,” 2018. [Online]. Available: <https://arxiv.org/abs/1803.08494>
- [72] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2211.10438>
- [73] J. A. Yang, J. Park, S. Sridharan, and P. T. P. Tang, “Training deep learning recommendation model with quantized collective communications,” in *Conference on Knowledge Discovery and Data Mining (KDD)*, 2020, p. 95.
- [74] Z. Zeng, X. Liu, M. Hang, X. Liu, Q. Zhou, C. Yang, Y. Liu, Y. Ruan, L. Chen, Y. Chen, Y. Hao, J. Xu, J. Nie, X. Liu, B. Zhang, W. Wen, S. Yuan, K. Wang, W.-Y. Chen, Y. Han, H. Li, C. Yang, B. Long, P. S. Yu, H. Tong, and J. Yang, “Interformer: Towards effective heterogeneous interaction learning for click-through rate prediction,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.09852>
- [75] D. Zha, L. Feng, L. Luo, B. Bhushanam, Z. Liu, Y. Hu, J. Nie, Y. Huang, Y. Tian, A. Kejariwal *et al.*, “Pre-train and search: Efficient embedding table sharding with pre-trained neural cost models,” *Proceedings of Machine Learning and Systems*, vol. 5, 2023.
- [76] J. Zhai, L. Liao, X. Liu, Y. Wang, R. Li, X. Cao, L. Gao, Z. Gong, F. Gu, J. He, Y. Lu, and Y. Shi, “Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 58 484–58 509. [Online]. Available: <https://proceedings.mlr.press/v235/zhai24a.html>
- [77] B. Zhang and R. Sennrich, “Root mean square layer normalization,” 2019. [Online]. Available: <https://arxiv.org/abs/1910.07467>
- [78] B. Zhang, L. Luo, Y. Chen, J. Nie, X. Liu, D. Guo, Y. Zhao, S. Li, Y. Hao, Y. Yao, G. Lakshminarayanan, E. D. Wen, J. Park, M. Naumov, and W. Chen, “Wukong: Towards a scaling law for large-scale recommendation,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.02545>
- [79] B. Zhang, L. Luo, X. Liu, J. Li, Z. Chen, W. Zhang, X. Wei, Y. Hao, M. Tsang, W. Wang, Y. Liu, H. Li, Y. Badr, J. Park, J. Yang, D. Mudigere, and E. Wen, “Dhen: A deep and hierarchical ensemble network for large-scale click-through rate prediction,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.11014>
- [80] W. Zhang, D. Li, C. Liang, F. Zhou, Z. Zhang, X. Wang, R. Li, Y. Zhou, Y. Huang, D. Liang, K. Wang, Z. Wang, Z. Chen, F. Wu, M. Chen, H. Li, Y. Wu, Z. Shu, M. Yuan, and S. Reddy, “Scaling user modeling: Large-scale online user representations for ads personalization in meta,” in *Companion Proceedings of the ACM Web Conference 2024*, ser. WWW ’24. ACM, May 2024, p. 47–55. [Online]. Available: <http://dx.doi.org/10.1145/3589335.3648301>
- [81] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer *et al.*, “Pytorch fsdp: experiences on scaling fully sharded data parallel,” *arXiv preprint arXiv:2304.11277*, 2023.